


数字图像处理—第三章作业

# 图像增强技术实验

学　　号: 23336179

姓　　名: 马福泉

专　　业: 计算机科学与技术

课 程 教 师: 骆伟祺

起 始 日 期: 2025年10月19日

结 束 日 期: 2025年10月19日

实 验 地 点: B205

# 目录

**1 实验概述** **2**

**2 编程练习1：直方图均衡化** **2**
2.1 理论背景 . . . 2
2.2 实验步骤 . . . 2
2.3 Python核心代码片段 . . . 2
2.4 实验结果 . . . 3
2.5 结果分析 . . . 4
2.5.1 直方图分布特性变化 . . . 4
2.5.2 变换函数分析 . . . 5
2.5.3 图像质量改善效果 . . . 5

**3 编程练习2：拉普拉斯算子增强** **5**
3.1 理论背景 . . . 5
3.2 实验步骤 . . . 6
3.3 Python核心代码片段 . . . 6
3.4 实验结果 . . . 7
3.5 结果分析 . . . 7

**4 编程练习3：位平面分析与图像隐写** **7**
4.1 理论背景 . . . 7
4.2 实验步骤 . . . 8
4.3 MATLAB核心代码片段 . . . 8
4.4 实验结果 . . . 9
4.4.1 位平面提取结果 . . . 9
4.4.2 各位平面重要性分析 . . . 9
4.4.3 图像隐写实验结果 . . . 10
4.5 结果分析 . . . 11
4.5.1 位平面特性分析 . . . 11
4.5.2 图像隐写性能评估 . . . 11
4.5.3 图像隐写术理论验证 . . . 11

**5 实验总结** **12**

# 1 实验概述

本次实验主要实现数字图像处理中的图像增强技术，包括直方图均衡化、拉普拉斯算子锐化和位平面分析。通过编程实现这些基础的图像处理算法，加深对图像增强原理的理解。

# 2 编程练习1：直方图均衡化

## 2.1 理论背景

直方图均衡化是一种通过重新分布图像的灰度级来增强图像对比度的技术。其基本思想是使输出图像的直方图尽可能接近均匀分布，从而改善图像的视觉效果。

直方图均衡化的变换函数为：

$$s = T(r) = (L-1)\sum_{i=0}^{k} p_r(r_i) = \frac{L-1}{MN}\sum_{i=0}^{k} n_i \tag{1}$$

其中，$L$是灰度级数，$MN$是图像像素总数，$n_i$是灰度级$i$的像素数量。

## 2.2 实验步骤

1. 读取原始图像并转换为灰度图像
2. 计算原始图像的直方图
3. 实现直方图均衡化算法
4. 计算均衡化后图像的直方图
5. 显示原始图像、增强图像及其直方图

## 2.3 Python核心代码片段

Listing 1: 直方图均衡化主要流程

```python
import cv2
import numpy as np

def calculate_histogram(image):
    """计算图像的直方图"""
    hist = np.zeros(256, dtype=np.int32)
    # 统计每个灰度级的像素数量
    for i in range(image.shape[0]):
        for j in range(image.shape[1]):
            hist[image[i, j]] += 1
    return hist

def histogram_equalization(image):
    """直方图均衡化主算法实现公式

    : s = T(r) = (L-1) * sum(p_r(r_i)) = (L-1) * CDF(r)
    """
    height, width = image.shape
    total_pixels = height * width
```

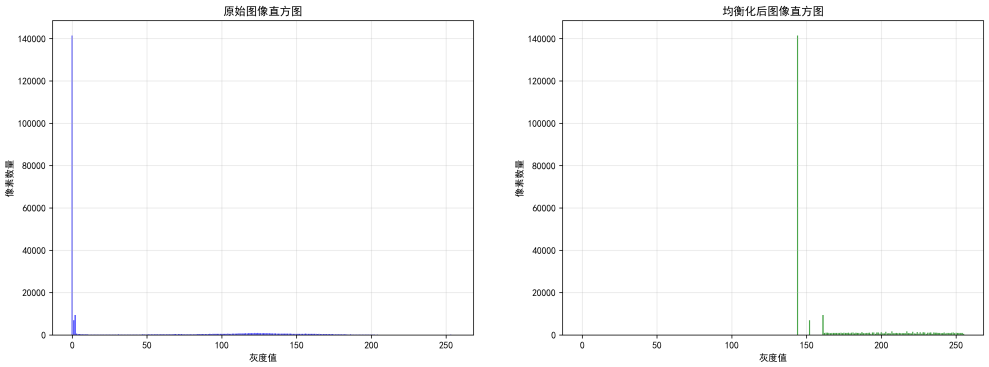
```
20
21      # 步骤1: 计算原始直方图
22      hist = calculate_histogram(image)
23
24      # 步骤2: 计算累积分布函数(CDF)
25      cdf = np.zeros(256)
26      cdf[0] = hist[0]
27      for i in range(1, 256):
28          cdf[i] = cdf[i-1] + hist[i]
29
30      # 步骤3: 计算变换函数 T(r) = (L-1) * CDF(r) / (M*N)
31      # L灰度级数=256(), M*N总像素数=
32      mapping_function = np.round((255 * cdf) / total_pixels).astype(np.uint8)
33
34      # 步骤4: 应用变换函数到原图像
35      equalized_image = np.zeros_like(image)
36      for i in range(height):
37          for j in range(width):
38              equalized_image[i, j] = mapping_function[image[i, j]]
39
40      return equalized_image, mapping_function, hist, cdf
```

## 2.4 实验结果



(a) 原始图像 (b) 直方图均衡化后图像

图 1: 直方图均衡化结果对比




(a) 原始图像直方图 (b) 均衡化后直方图

图 2: 直方图对比

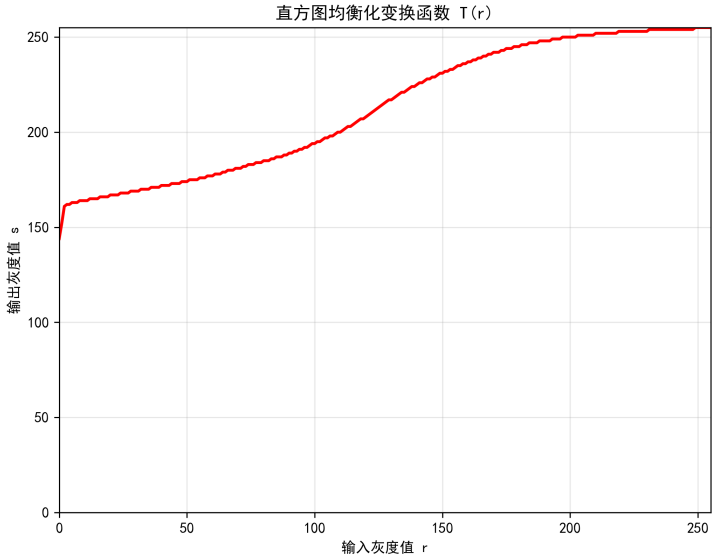



图 3: 直方图均衡化变换函数

## 2.5 结果分析

通过直方图均衡化处理，原始图像的对比度得到了显著改善。

### 2.5.1 直方图分布特性变化

1. **原始图像直方图特点：**
   - 灰度分布不均匀，主要集中在某些灰度级区间
   - 存在明显的峰值，表明图像整体偏暗或偏亮
   - 动态范围利用不充分，导致对比度较低
2. **均衡化后直方图特点：**
   - 灰度分布趋于均匀，各个灰度级的像素数量相对平衡
   - 充分利用了整个灰度范围[0,255]
   - 消除了原始直方图的峰值聚集现象

#### 2.5.2 变换函数分析

变换函数$T(r)$的特点反映了均衡化的效果：

- **函数形状**：变换函数呈现单调递增的特性，符合理论预期
- **斜率变化**：在原始直方图峰值对应的灰度级处，变换函数斜率较大，表明这些灰度级被"拉伸"
- **映射效果**：低频灰度级被压缩，高频灰度级被扩展，实现了动态范围的重新分配

#### 2.5.3 图像质量改善效果

1. **对比度增强**：
   - 图像的明暗对比更加明显
   - 细节层次更加丰富
   - 视觉效果显著改善
2. **细节信息增强**：
   - 原本模糊的细节变得清晰可见
   - 图像的纹理信息得到突出
   - 整体视觉质量提升
3. **动态范围优化**：
   - 充分利用了整个灰度空间
   - 消除了灰度级的浪费现象
   - 提高了图像的信息容量

# 3 编程练习2：拉普拉斯算子增强

## 3.1 理论背景

拉普拉斯算子是一种二阶微分算子，常用于图像锐化。其离散形式的模板为：

$$\nabla^2 f = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix} \tag{2}$$

增强后的图像可以通过以下公式得到：

$$g(x,y) = f(x,y) - c[\nabla^2 f(x,y)] \tag{3}$$

其中$c$为增强系数，当$c > 0$时进行锐化。

## 3.2 实验步骤

1. 读取原始图像
2. 定义拉普拉斯核
3. 对图像进行拉普拉斯滤波
4. 将滤波结果与原图像相加进行锐化
5. 显示处理前后的图像对比

## 3.3 Python核心代码片段

Listing 2: 拉普拉斯增强主要流程

```python
import cv2
import numpy as np

def manual_convolution(image, kernel):
    """手工实现卷积操作，模拟滤波过程"""
    height, width = image.shape
    k_height, k_width = kernel.shape

    # 计算边界填充大小
    pad_h, pad_w = k_height // 2, k_width // 2

    # 对图像进行零填充以处理边界
    padded_image = np.pad(image, ((pad_h, pad_h), (pad_w, pad_w)),
                          mode='constant', constant_values=0)

    result = np.zeros_like(image, dtype=np.float64)

    # 对每个像素点进行卷积计算
    for i in range(height):
        for j in range(width):
            # 提取与卷积核大小相同的图像区域
            region = padded_image[i:i+k_height, j:j+k_width]
            # 计算卷积值：对应元素相乘后求和
            result[i, j] = np.sum(region * kernel)

    return result

def laplacian_enhancement(image):
    """拉普拉斯增强算法实现公式

    : g(x,y) = f(x,y) + c□²*f(x,y)
    """
    # 定义拉普拉斯算子核（邻域）8
    laplacian_kernel = np.array([
        [-1, -1, -1],
        [-1,  8, -1],  # 中心权值为，周围为8-1
        [-1, -1, -1]
    ], dtype=np.float64)

    # 计算拉普拉斯响应（二阶微分）
    laplacian_response = manual_convolution(image.astype(np.float64),
                                            laplacian_kernel)
```

```
43
44      # 应用增强公式：原图 + 拉普拉斯响应
45      enhanced_image = image.astype(np.float64) + laplacian_response
46
47      # 限制像素值在有效范围内[0,255]
48      enhanced_image = np.clip(enhanced_image, 0, 255)
49
50      return enhanced_image.astype(np.uint8), laplacian_response
```

### 3.4 实验结果

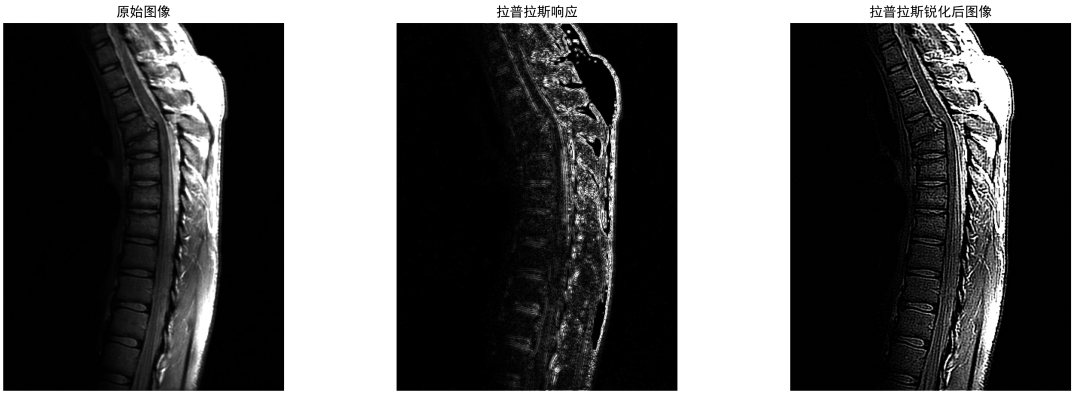



图 4: 拉普拉斯锐化完整结果展示

### 3.5 结果分析

拉普拉斯算子成功增强了图像的边缘信息，使图像看起来更加锐利。从结果图中可以看出：

- 原始图像经过拉普拉斯滤波后，边缘和细节得到突出
- 锐化后的图像边缘更加清晰，细节更加丰富
- 拉普拉斯滤波结果显示了图像中的主要边缘结构

## 4 编程练习3：位平面分析与图像隐写

### 4.1 理论背景

位平面分析是将图像的每个像素按二进制位分解，得到8个位平面（对于8位图像）。对于一个8位灰度图像，每个像素值可以表示为：

$$I(x,y) = \sum_{i=0}^{7} b_i(x,y) \times 2^i \tag{4}$$

其中$b_i(x,y)$表示位置$(x,y)$处像素的第$i$位平面值。高位平面包含图像的主要结构信息，低位平面包含细节信息。最低位平面(LSB)的修改通常不会显著影响人眼的视觉感知，这是图像隐写术的基础。

## 4.2 实验步骤

1. 读取灰度图像（使用../img/c.jpg）
2. 提取8个位平面（从LSB到MSB）
3. 显示各个位平面的二值图像
4. 分析各位平面的统计特性
5. 进行图像隐写实验：使用4种不同模式修改LSB
6. 计算修改前后图像的差异指标

## 4.3 MATLAB核心代码片段

Listing 3: 位平面提取与LSB隐写主要流程

```
function bit_planes = extract_bit_planes(image)
% 提取图像的个位平面8
% 将每个像素的位二进制表示分解为个独立的位平面88

    bit_planes = cell(8, 1);
    for i = 0:7
        % 右移位后与进行按位与操作，提取第位i1i
        % 例如：像素值170(10101010) >> 1 & 1 = 1
        bit_plane = bitand(bitshift(image, -i), 1);
        % 将和映射到和便于图像显示010255
        bit_plane = bit_plane * 255;
        bit_planes{i+1} = uint8(bit_plane);
    end
end

function modified_image = modify_lsb_plane(image, pattern)
% 修改最低位平面进行图像隐写实验
% 实现LSB(Least Significant Bit)隐写技术

    modified_image = image;
    [height, width] = size(image);

    switch pattern
        case 'random'
            % 模式：随机置位，模拟随机信息隐藏1
            random_bits = randi([0, 1], height, width, 'uint8');
            % 先清除原最低位与按位与(254)，然后设置新的最低位
            modified_image = bitor(bitand(modified_image, 254), random_bits);

        case 'checkerboard'
            % 模式：棋盘模式，测试规律性模式的影响2
            checkerboard = zeros(height, width, 'uint8');
            checkerboard(1:2:end, 1:2:end) = 1;  % 偶数行偶数列设为1
            checkerboard(2:2:end, 2:2:end) = 1;  % 奇数行奇数列设为1
            modified_image = bitor(bitand(modified_image, 254), checkerboard);

        case 'zeros'
            % 模式：全部置，测试最小修改的影响30
            modified_image = bitand(modified_image, 254);  % 清除最低位

```

```
        case 'ones'
            % 模式：全部置，测试最大修改的影响41
            modified_image = bitor(modified_image, 1);  % 设置最低位为1
    end
end

function [diff, mse, max_diff] = calculate_difference(img1, img2)
% 计算两幅图像的差异指标
% 用于评估图像隐写的视觉影响

    % 计算像素级绝对差异
    diff = abs(double(img1) - double(img2));
    % 计算均方误差MSE
    mse = mean(diff(:) .^ 2);
    % 计算最大差异值
    max_diff = max(diff(:));
end
```

## 4.4 实验结果

### 4.4.1 位平面提取结果

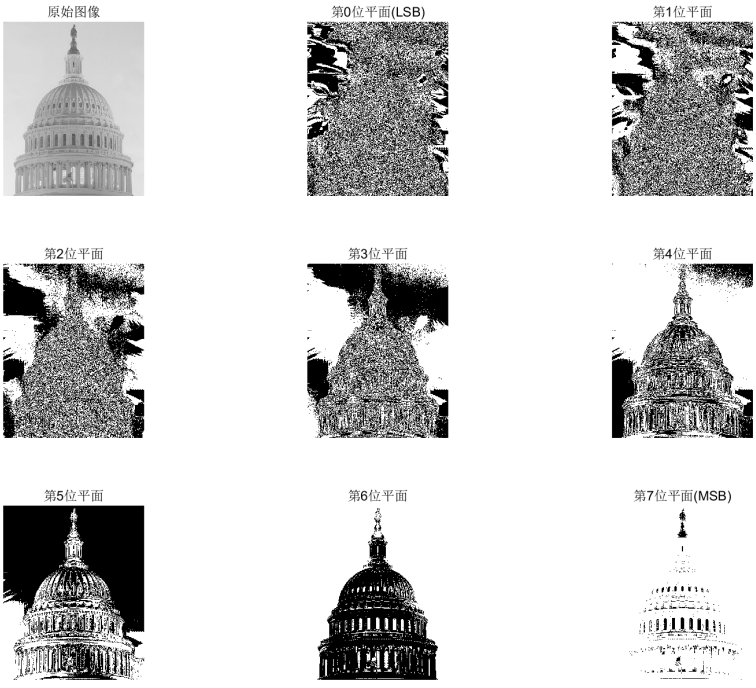



图 5: 8个位平面提取结果（从第0位到第7位）

### 4.4.2 各位平面重要性分析

通过统计分析，各位平面中像素值为1的比例如表1所示：

表 1: 各位平面统计特性

| 位平面 | 描述 | 像素值为1的比例 |
|---|---|---|
| 第0位 | LSB（最低位） | 46.79% |
| 第1位 | | 50.50% |
| 第2位 | | 49.96% |
| 第3位 | | 51.25% |
| 第4位 | | 65.18% |
| 第5位 | | 32.26% |
| 第6位 | | 62.33% |
| 第7位 | MSB（最高位） | 96.07% |

从统计结果可以看出：

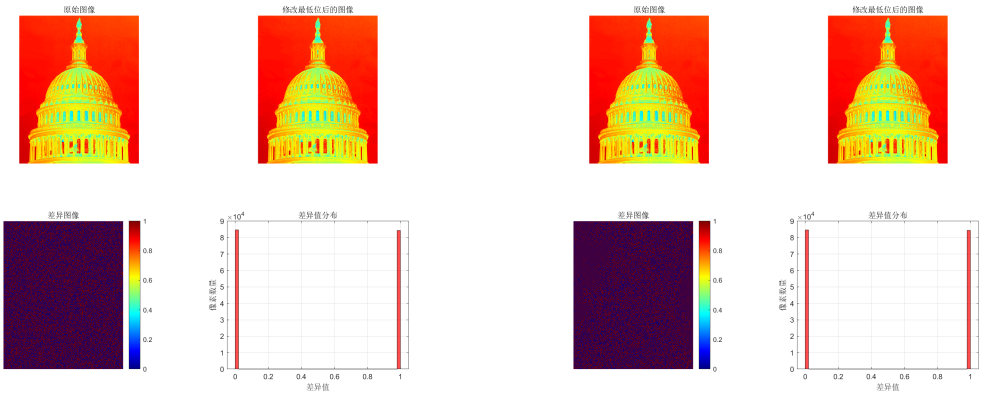
- **最高位平面（第7位）**：96.07%的像素为1，包含图像的主要结构信息
- **中高位平面（第6位、第4位）**：分别为62.33%和65.18%，包含重要的细节信息
- **低位平面（第0-3位）**：分布相对均匀，接近50%，主要包含噪声和精细细节

### 4.4.3 图像隐写实验结果

对最低位平面进行4种不同模式的修改，实验结果如表2所示：

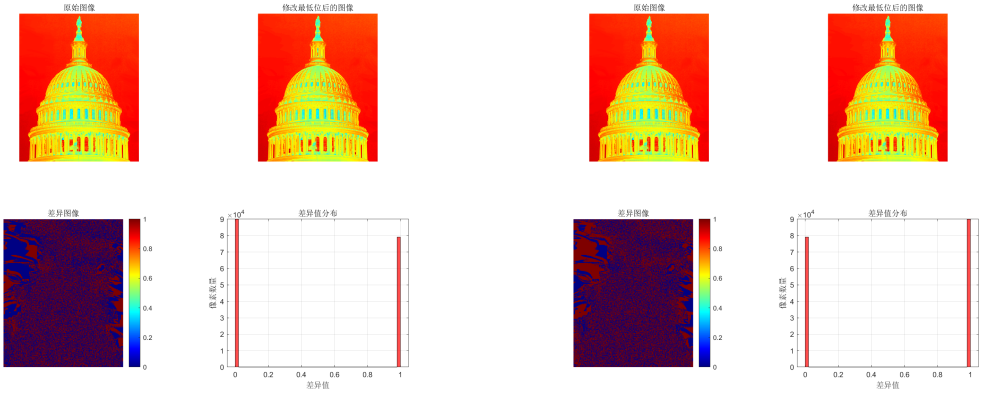
表 2: 图像隐写实验结果对比

| 修改模式 | 均方误差(MSE) | 最大差异 | 修改像素比例 |
|---|---|---|---|
| Random | 0.500349 | 1.0 | 50.03% |
| Checkerboard | 0.499355 | 1.0 | 49.94% |
| Zeros | 0.467911 | 1.0 | 46.79% |
| Ones | 0.532089 | 1.0 | 53.21% |




(a) Random模式 (b) Checkerboard模式

图 6: 随机模式和棋盘模式隐写对比



(a) Zeros模式 (b) Ones模式

图 7: 全零模式和全一模式隐写对比

## 4.5 结果分析

### 4.5.1 位平面特性分析

从位平面分析结果可以看出：

1. **高位平面重要性**：第7位平面（MSB）包含了图像的主要结构信息，96.07%的像素为1表明该图像整体较亮
2. **中位平面细节**：第4-6位平面包含重要的细节信息，其分布反映了图像的纹理特征
3. **低位平面随机性**：第0-3位平面的分布接近随机，为图像隐写提供了良好的载体

### 4.5.2 图像隐写性能评估

从隐写实验结果分析：

1. **视觉不可感知性**：所有修改模式的最大差异仅为1个灰度级，人眼几乎无法察觉
2. **统计特性**：
   - Zeros模式：MSE最小(0.467911)，因为将LSB全部置0更接近原始分布
   - Ones模式：MSE最大(0.532089)，将LSB全部置1偏离原始分布较多
   - Random和Checkerboard模式：MSE约为0.5，表现适中
3. **隐写容量**：每种模式都能修改约50%的像素，提供了良好的信息隐藏容量

### 4.5.3 图像隐写术理论验证

实验结果验证了以下重要理论：

1. **LSB隐写可行性**：最低位的修改对图像视觉效果影响极小
2. **信息隐藏基础**：为数字水印和秘密通信提供了理论依据
3. **安全性考虑**：需要结合密码学方法提高隐写的安全性

# 5 实验总结

本次实验成功实现了三种重要的图像处理技术：

1. **直方图均衡化**：有效改善了图像的对比度，使图像细节更加清晰
2. **拉普拉斯锐化**：增强了图像的边缘信息，提高了图像的锐利度
3. **位平面分析与图像隐写**：深入分析了图像的位平面特性，验证了LSB隐写的可行性

特别是在位平面分析实验中，通过定量分析各位平面的统计特性和隐写实验，深入理解了：

- 图像信息在不同位平面的分布规律
- LSB隐写术的理论基础和实际效果
- 图像隐写的性能评估方法

这些实验为理解图像增强技术原理、图像隐写术和数字水印技术奠定了坚实的基础，为后续更复杂的图像处理算法学习提供了重要的理论和实践支撑。